

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4382

Калинина А.В
Смирнов Н.А
Бурая В.С.
Писаренко А.А.

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Калинина А.В.

Группа: 4382

Тема практики: UI-тестирование сайта Stepik

Задание на практику:

Проектирование и реализация программного комплекса для автоматизированного UI-тестирования функциональных блоков веб-приложения с использованием современного технологического стека. Работа включает в себя разработку подробного чек-листа из 10 тестовых сценариев общим объемом не менее 50 действий, а также их программную реализацию на языке Java с применением библиотек Selenide и JUnit 5. Архитектура проекта строится на принципах Page Object Model и компонентно-ориентированного подхода, что обеспечивает четкое разделение логики тестов и структуры страниц для удобства поддержки кода. Проект также предусматривает настройку системы логирования Log4j2 для мониторинга выполнения тестов, организацию совместной разработки в репозитории GitHub и подготовку итоговой отчетности с описанием архитектуры классов и построением соответствующих UML-диаграмм.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты

Руководитель

Калинина А.В.
Смирнов Н.А.
Бурая В.С.
Писаренко А.А.

А.М. Шевелёва

АННОТАЦИЯ

Данный отчёт посвящён проектированию и программной реализации масштабируемой архитектуры автоматизированного тестирования для образовательной платформы Stepik. В ходе работы был проведён анализ функциональных возможностей системы и сформирован чек-лист из десяти ключевых пользовательских сценариев. На языке Java разработан тестовый фреймворк, базирующийся на паттерне Page Object Model, что обеспечило изоляцию логики тестов от деталей реализации интерфейса. Программная реализация выполнена с использованием библиотек Selenide и JUnit 5, интегрирована система логирования Log4j2 и подготовлена техническая документация, включая UML-диаграммы классов. Итоговый проект размещён в репозитории GitHub.

SUMMARY

This report focuses on the design and software implementation of a scalable automated testing architecture for the Stepik educational platform. In the course of the work, an analysis of the system's functionality was carried out and a checklist of ten key user scenarios was formed. A test framework based on the Page Object Model pattern and a component-based approach has been developed in Java, which ensures that the test logic is isolated from the interface implementation details. The software implementation was performed using the Selenide and JUnit 5 libraries, the Log4j2 logging system was integrated, and technical documentation was prepared, including UML class diagrams. The final project is hosted in the GitHub repository.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	23
2.1. Описание программных классов и их методов	23
2.2. UML-диаграмма классов	25
2.3. Реализация взаимодействия компонентов.....	26
3 ИНДИВИДУАЛЬНАЯ РАБОТА	28
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	31
5 ОТЗЫВ РУКОВОДИТЕЛЯ	32
ЗАКЛЮЧЕНИЕ	33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизированное функциональное тестирование пользовательских интерфейсов (UI) современного веб-приложения. Ручная проверка взаимодействия пользователя с элементами системы является трудозатратной и подверженной ошибкам. Автоматизация тестирования таких компонентов, как системы поиска, многоуровневые фильтры и навигационные блоки, позволяет обеспечить стабильность работы критически важного функционала и гарантировать корректность отображения данных при внесении изменений в код.

Целью практики является проектирование и программная реализация масштабируемой архитектуры автоматизированного тестирования на базе объектно-ориентированного подхода. Основной упор делается на создание гибкого тестового фреймворка, который минимизирует дублирование кода, упрощает поддержку тестов и обеспечивает высокую надежность проверок за счет использования современных инструментов автоматизации и проверенных архитектурных паттернов.

Для достижения поставленной цели необходимо решить следующие задачи:

Провести детальный анализ функциональных возможностей тестируемой платформы и сформировать чек-лист из десяти сценариев, покрывающих ключевые пользовательские пути.

Разработать программную архитектуру проекта на языке Java, используя паттерн Page Object Model [1] и компонентно-ориентированный подход для изоляции логики тестов от деталей реализации интерфейса.

Осуществить программную реализацию тестов с применением библиотек Selenide [2] и JUnit5 [3], интегрировать систему логирования Log4j2 [4] для мониторинга выполнения проверок, а также подготовить

техническую документацию, включающую визуализацию структуры системы с помощью UML-диаграмм классов.

Вклад каждого участника:

Калинина А.В.: реализация Page Objects (HomePage, SearchResultsPage, CoursePage, ProfilePages), создание чек-листа, настройка логирования, подготовка и финализация README, приведение классов к единому порядку, вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов, добавление документации к классам pages и BaseTest.

Смирнов Н.А.: Архитектура проекта и базовая настройка: создание Maven проекта, подключение зависимостей (Selenide, JUnit5, Log4j2), реализация папки components (SearchComponent, FilterComponent, CourseCardItem и т.д.), настройка BaseTest, инициализация структуры проекта, работа над чек-листом, проверка архитектуры кода и проверка соблюдения принципов Page Object Model, приведение классов к единому порядку, вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов.

Бурая В.С.: Тесты №1–5: навигация по страницам, поиск с фильтрами, фильтрация по ценовому диапазону, поиск для продвинутых с сертификатом, сброс текстового поиска "крестиком". Проверка стабильности выполнения тестов. Исправление и добавление методов. Удаление всех комментариев с действиями из тестов. Вынесение тестов с фильтрацией в отдельный класс.

Писаренко А.А.: Тесты №6–10: смена запроса с сохранением фильтра, несуществующий запрос с фильтрами, поиск по автору с переходом в профиль, фильтрация акционных курсов, добавление курса в избранное из поиска. Добавление assert-проверок и обработка исключений. Проверка работы всех тестов в связке. Исправление и добавление некоторых методов. Создание отдельного класса для работы с окнами. Вынесение всех магических чисел в константы.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 10 тестов:

1. Текстовый поиск и навигация по страницам
2. Поиск с фильтрами
3. Фильтрация по ценовому диапазону
4. Поиск для продвинутых с сертификатом
5. Сброс текстового поиска «крестиком»
6. Смена запроса с сохранением фильтра
7. Несуществующий запрос с фильтрами
8. Поиск по автору с переходом в профиль
9. Фильтрация акционных курсов
10. Добавление курса в избранное из поиска

Тест №1: Текстовый поиск и навигация по страницам

Данный тест проверяет базовую работоспособность поиска по ключевому слову, корректный переход между страницами результатов через пагинацию и возможность открытия детальной страницы курса.

Результаты действий представлены на рисунках (см. рисунки 1 – 4).

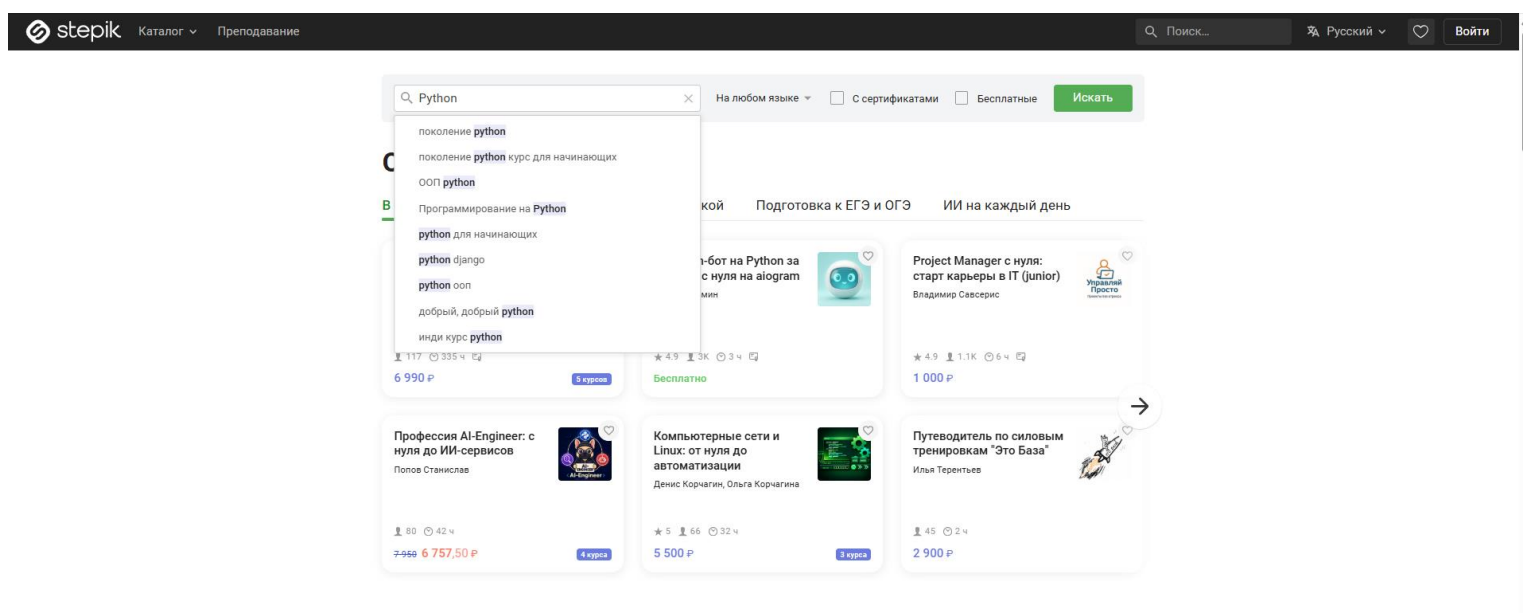


Рисунок 1 – Введение в поиск запрос «Python»

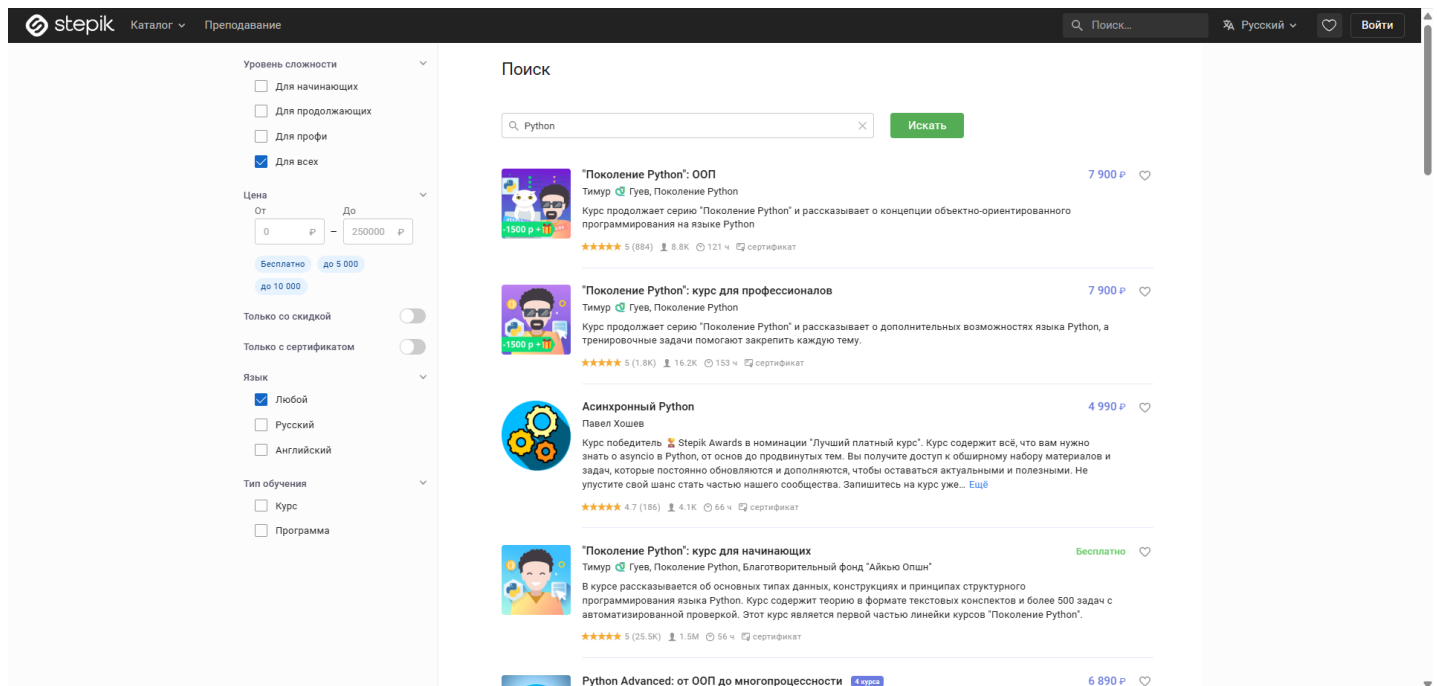


Рисунок 2 – Результаты поиска по запросу «Python»

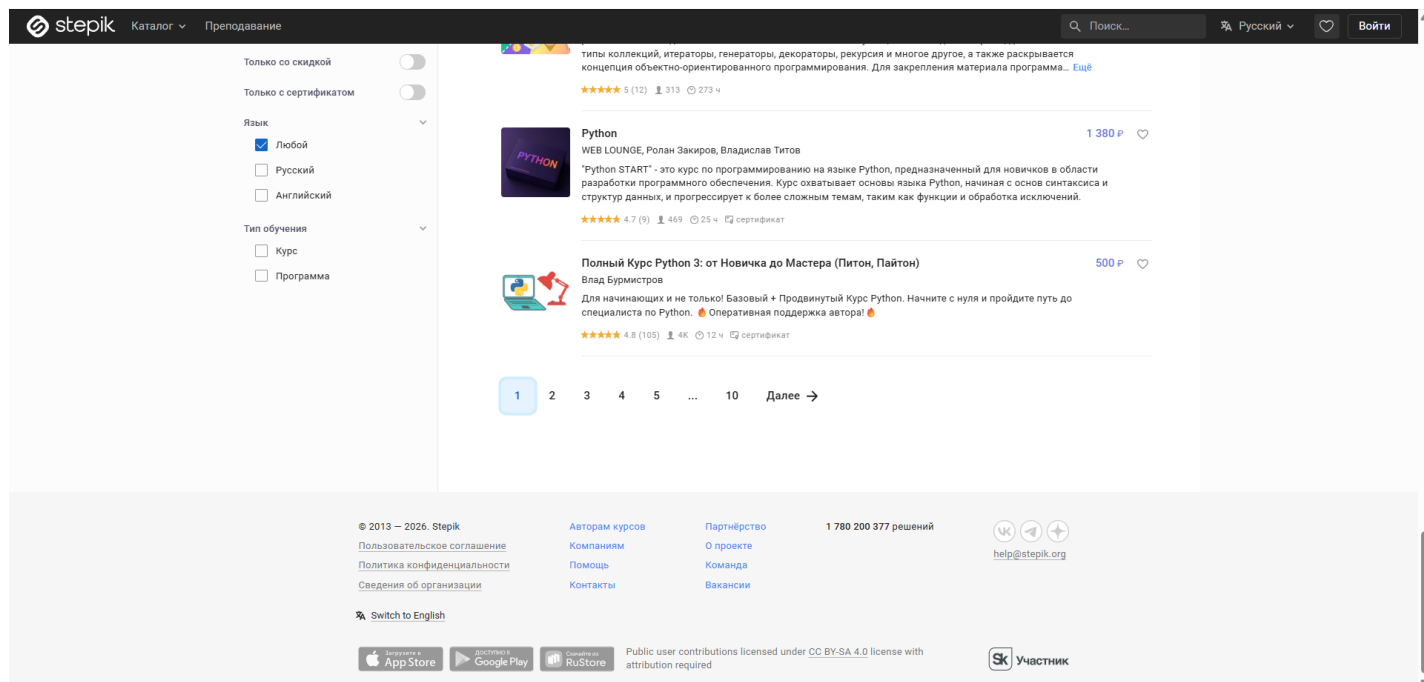


Рисунок 3 – Прокрутка вниз к пагинации

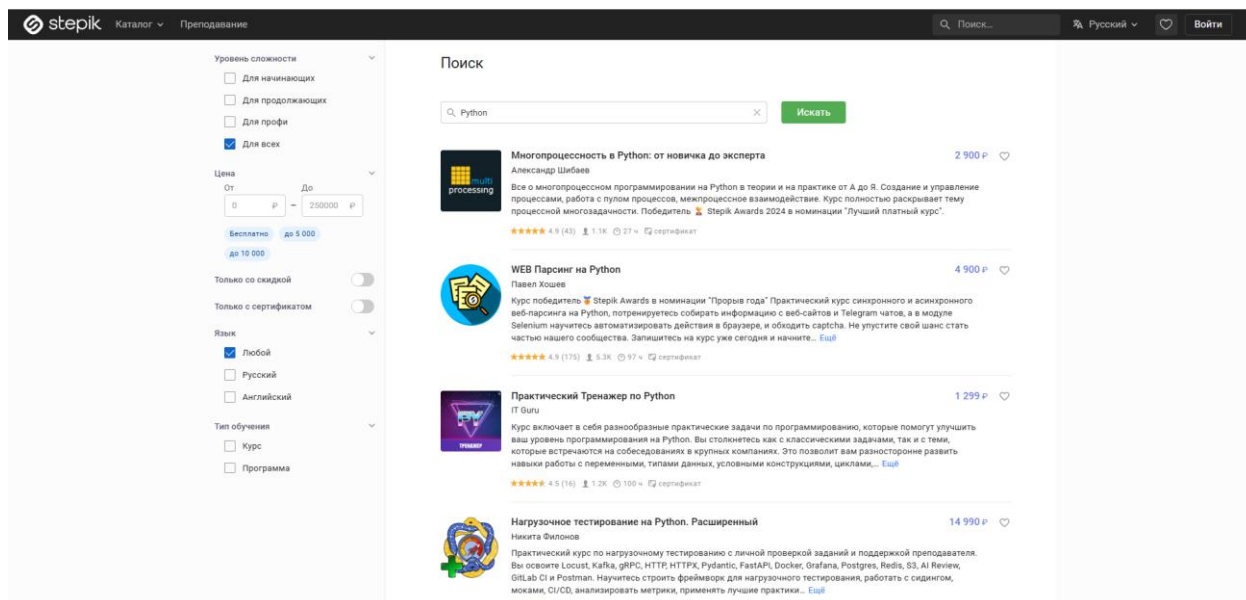


Рисунок 4 – Переход на вторую страницу запросов

Тест №2: Поиск с фильтрами

Данный тест верифицирует работу системы фильтрации при поиске, проверяя корректность применения фильтров по языку обучения и типу курса (специализации).

Результаты действий представлены на рисунках (см. рисунки 5, 6).

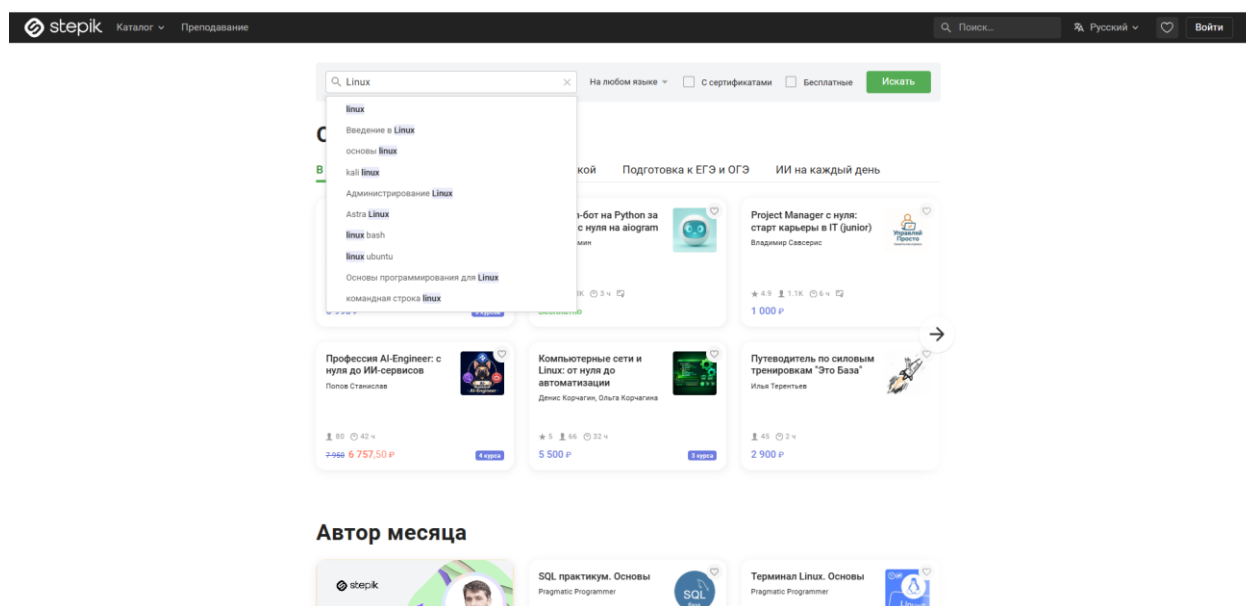


Рисунок 5 – Введение в поиск запрос «Linux»

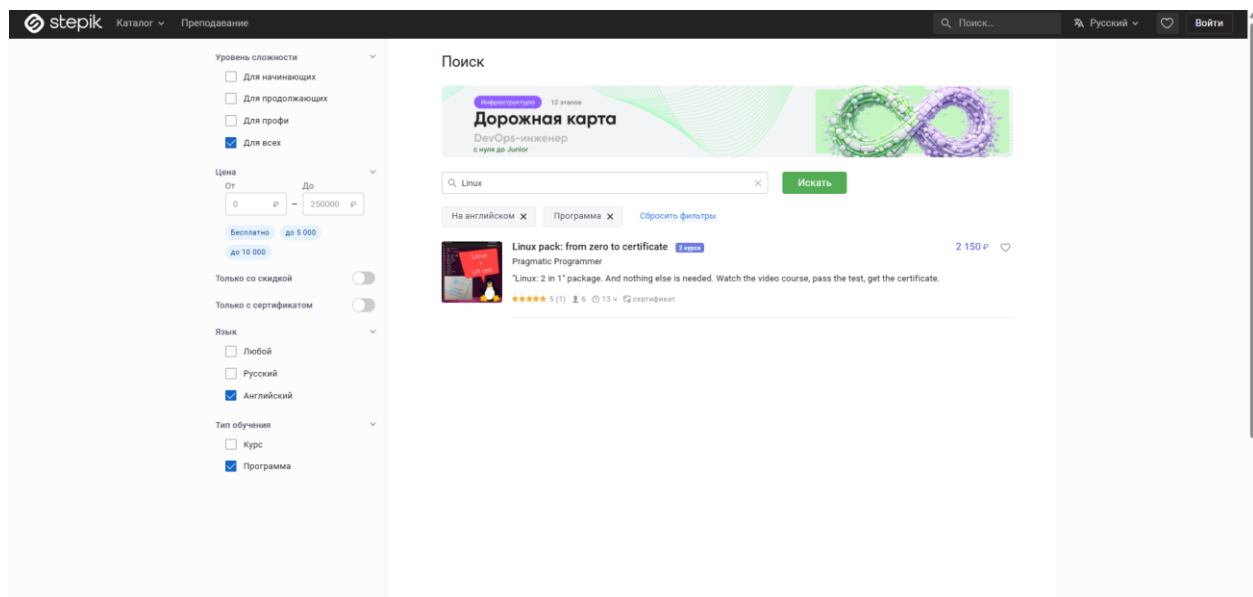


Рисунок 6 – Применение чекбоксов «Английский» и «Программа»

Тест №3: Фильтрация по ценовому диапазону

Данный тест подтверждает возможность подбора курсов в строго заданных границах стоимости, проверяя работу полей ввода минимальной и максимальной цены.

Результаты действий представлены на рисунках (см. рисунки 7, 8).

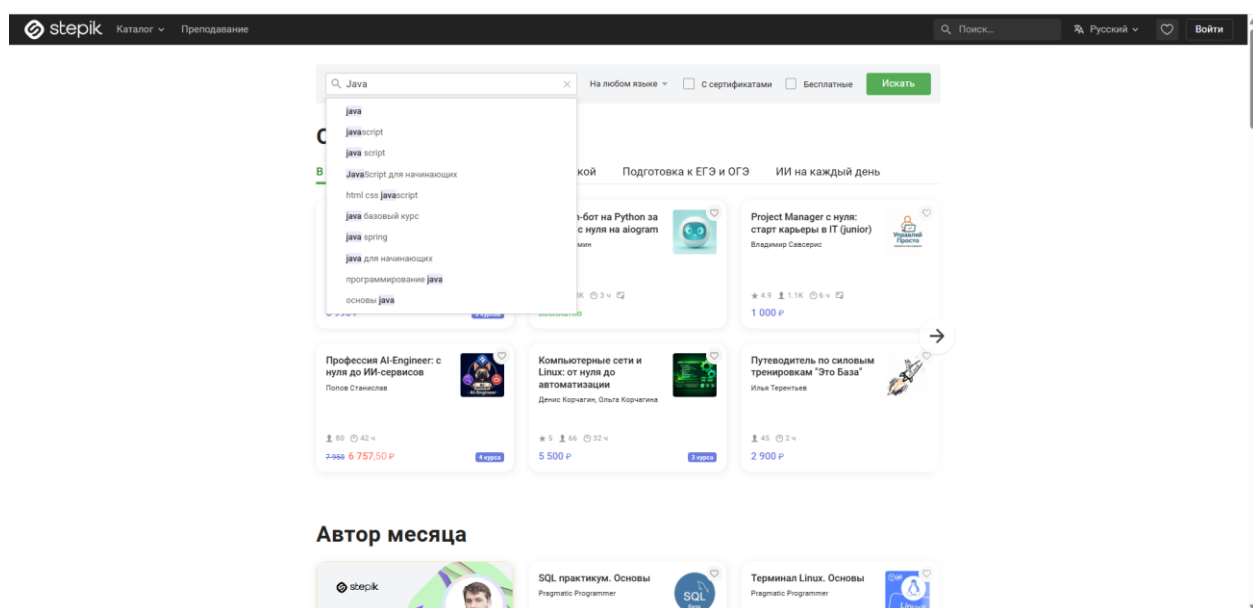


Рисунок 7 – Введение в поиск запрос «Java»

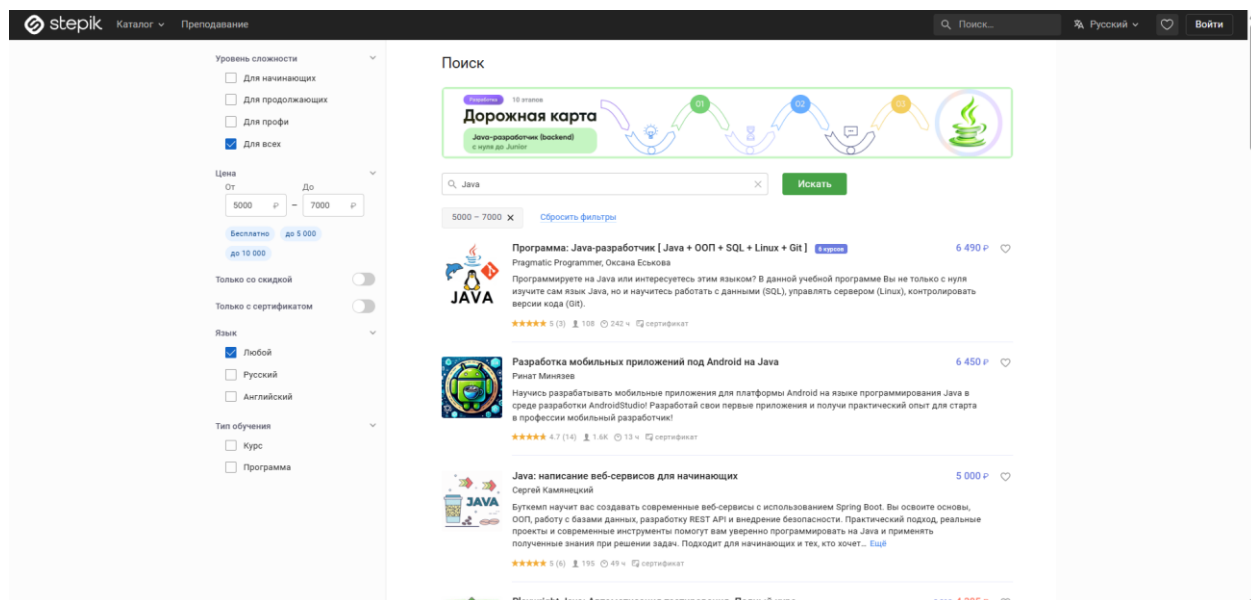


Рисунок 8 – Применение фильтра по цене

Тест №4: Поиск для продвинутых с сертификатом

Данный тест имитирует запрос профессионального пользователя, проверяя одновременную работу фильтра высокого уровня сложности и тумблера поиска курсов только с выдачей сертификата.

Результаты действий представлены на рисунках (см. рисунки 9 – 10).

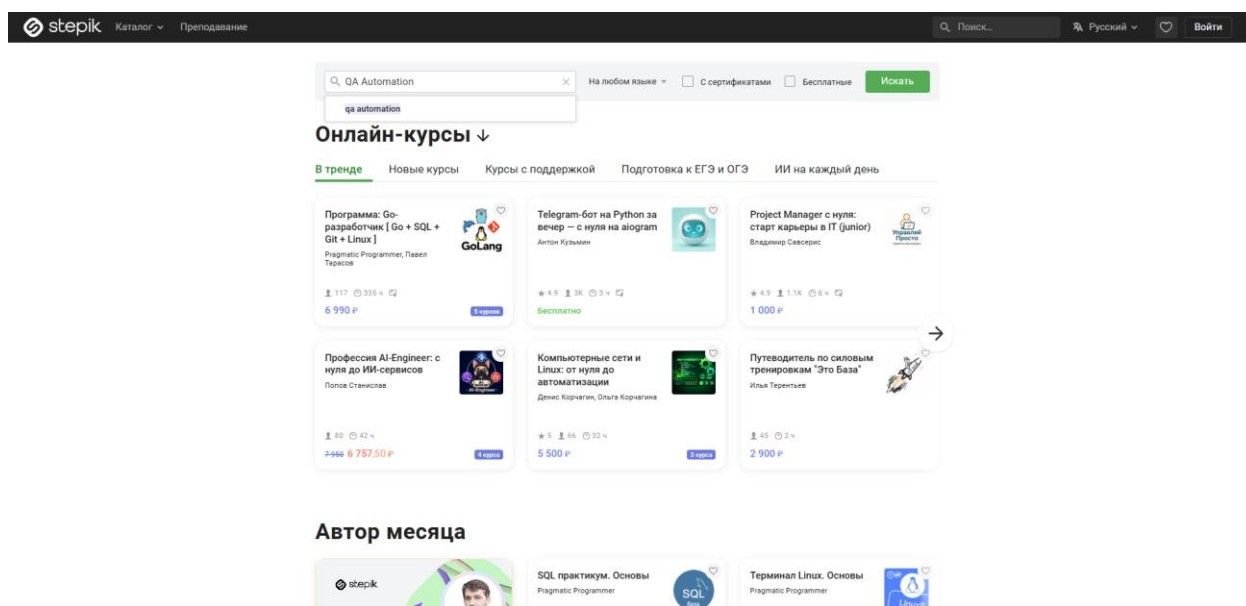


Рисунок 9 – Введение в поиск запрос «QA Automation»

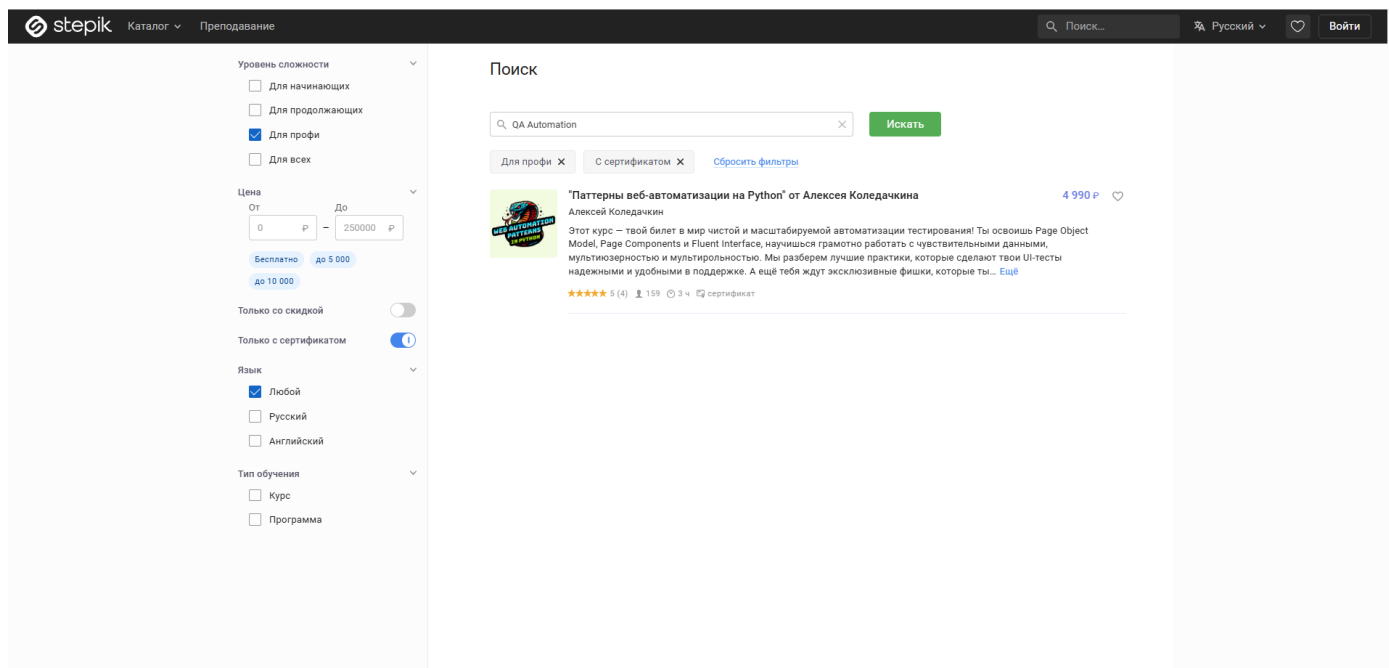


Рисунок 10 – Применение чекбоксов «Для профи» и «Только с сертификатом»

Тест №5: Сброс текстового поиска «крестиком»

Данный тест проверяет функциональность элемента очистки поисковой строки, контролируя удаление текста запроса и автоматический возврат каталога к общему виду.

Результаты действий представлены на рисунках (см. рисунки 11 - 13).

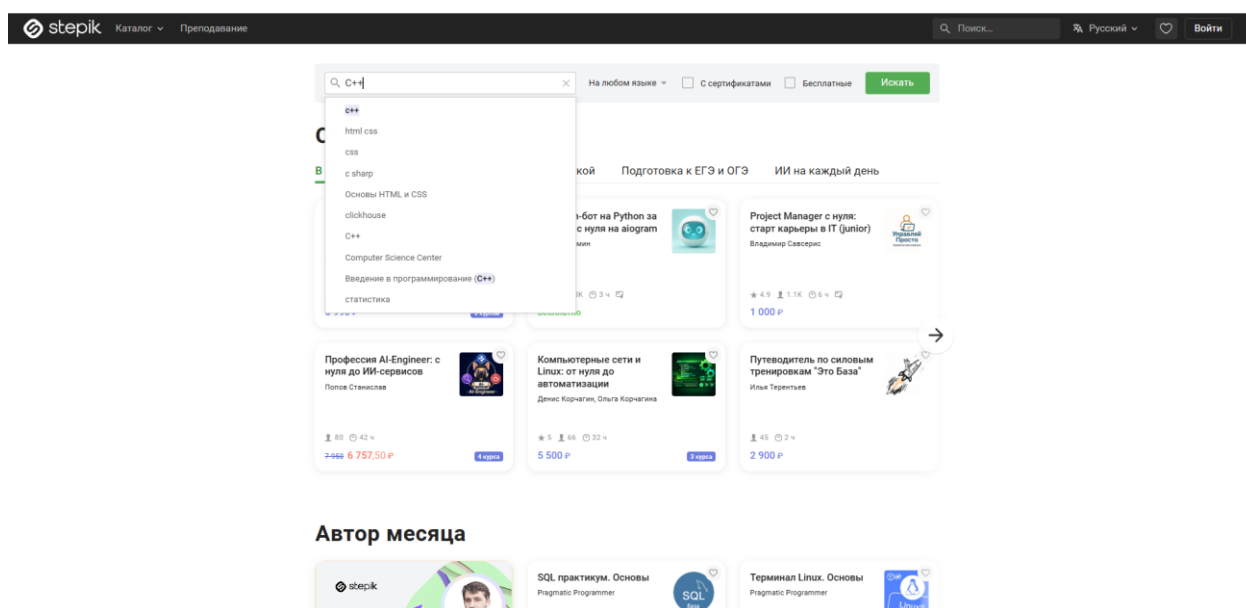


Рисунок 11 – Введение в поиск запрос «C++»

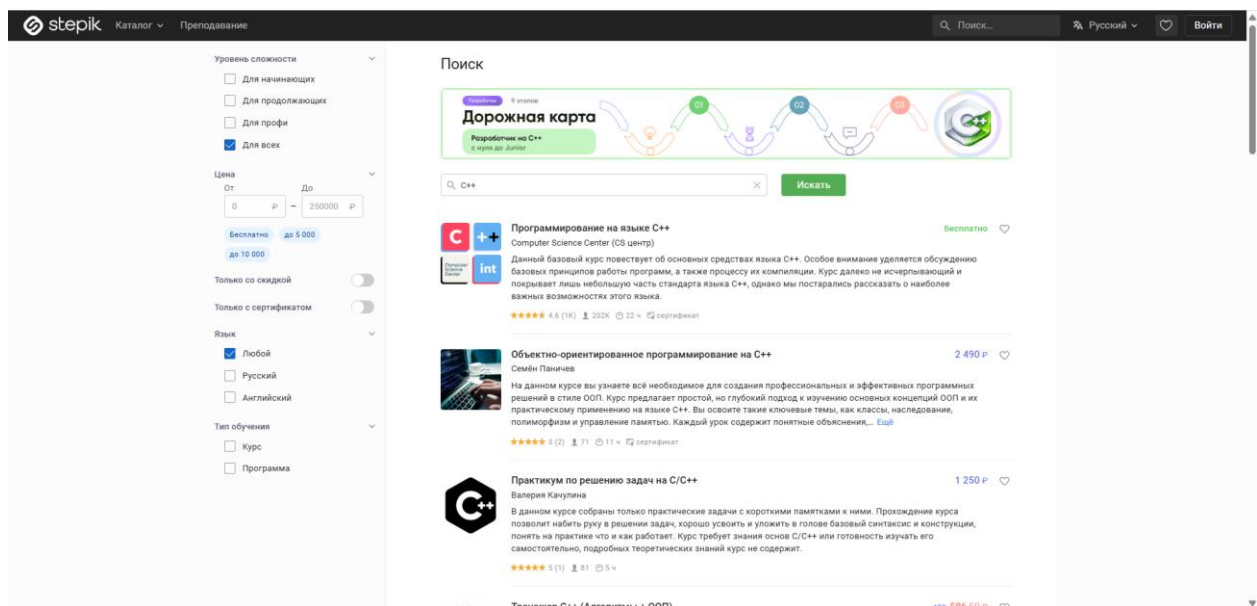


Рисунок 12 – Вывод всех доступных курсов

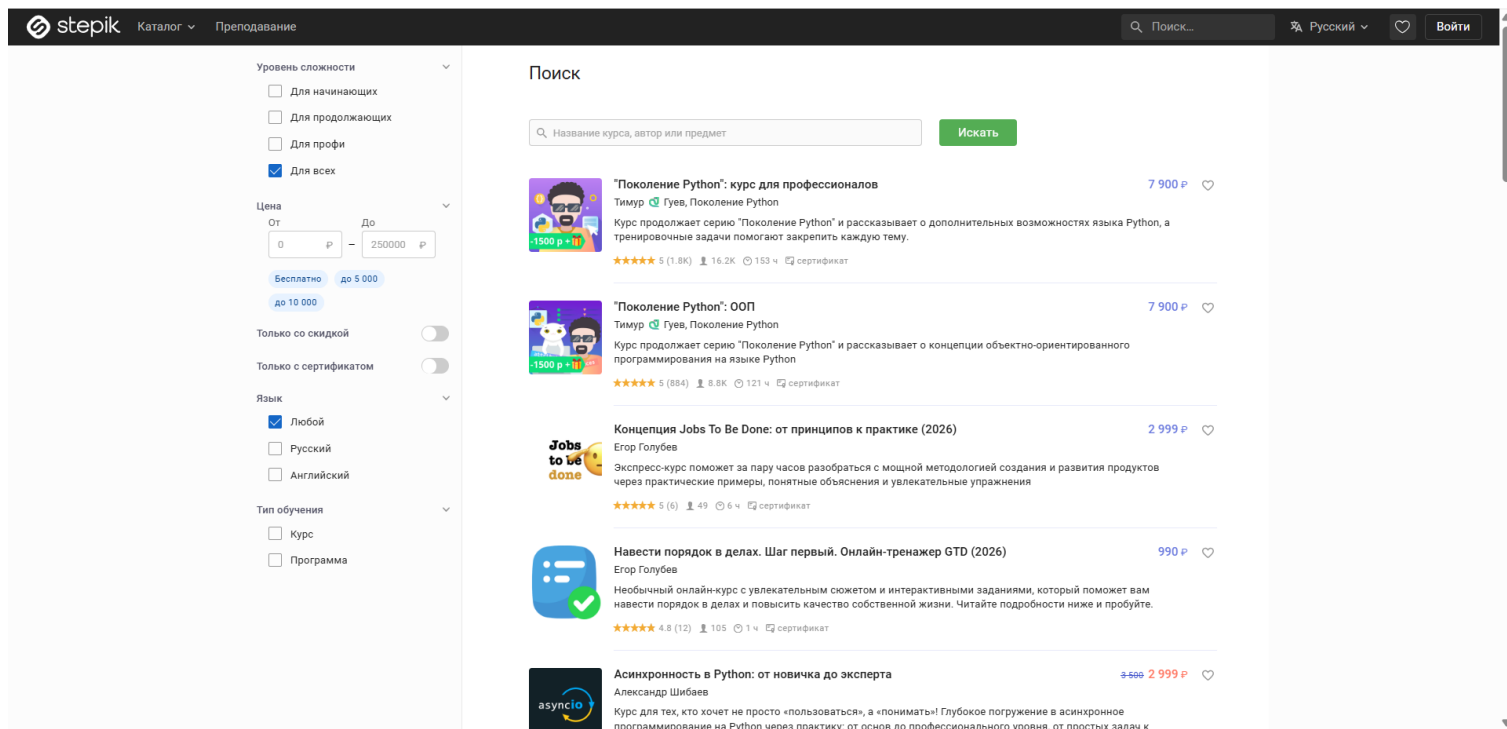


Рисунок 13 – Нажатие крестика

Тест №6: Смена запроса с сохранением фильтра

Данный тест работает с интеллектуальным поведением интерфейса, проверяя, остаются ли активными выбранные фильтры (например, язык) при вводе нового ключевого слова в поле поиска.

Результаты действий представлены на рисунках (см. рисунки 14 – 16).

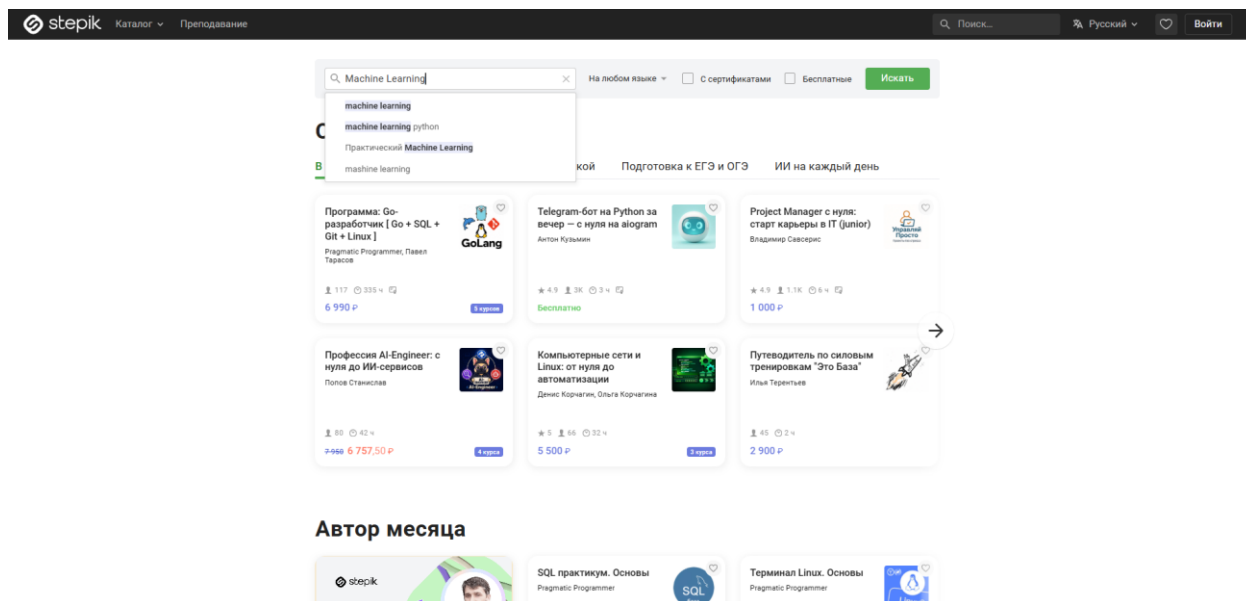


Рисунок 14 – Введение в поиск запрос «Machine Learning»

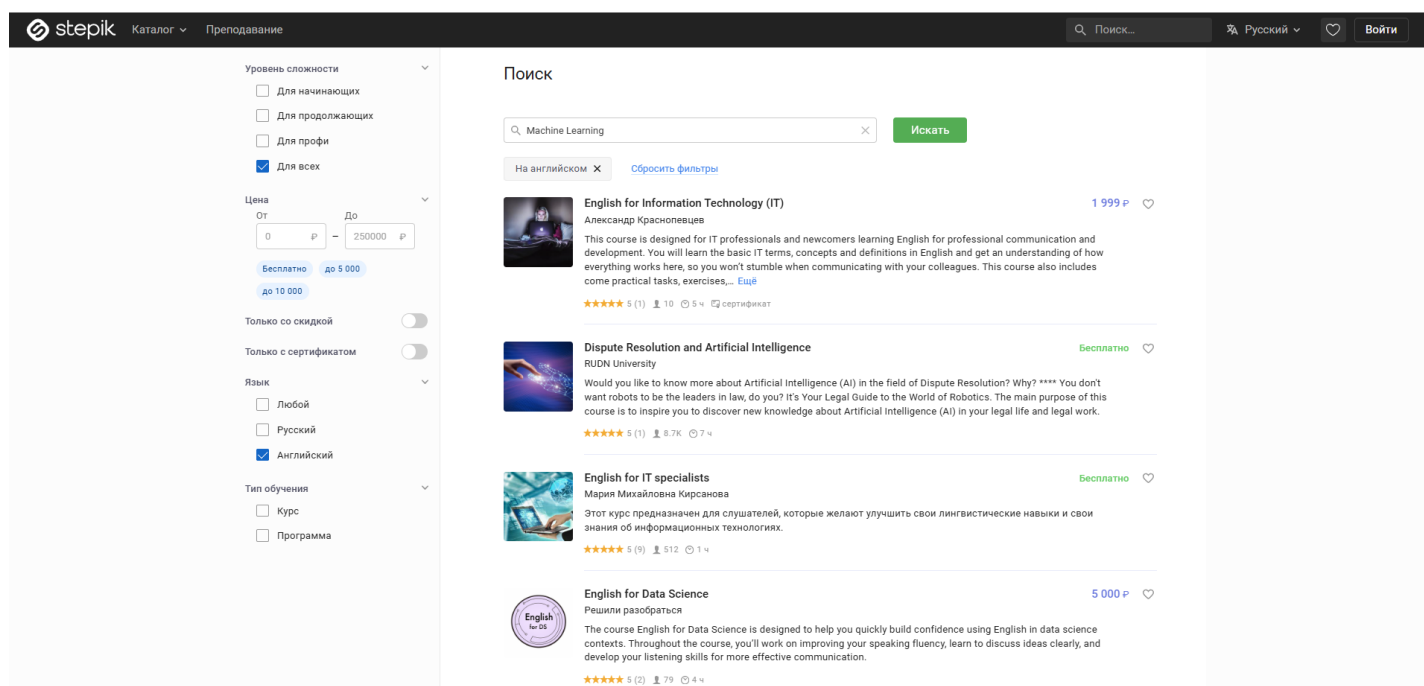


Рисунок 15 – Показ всех доступных курсов

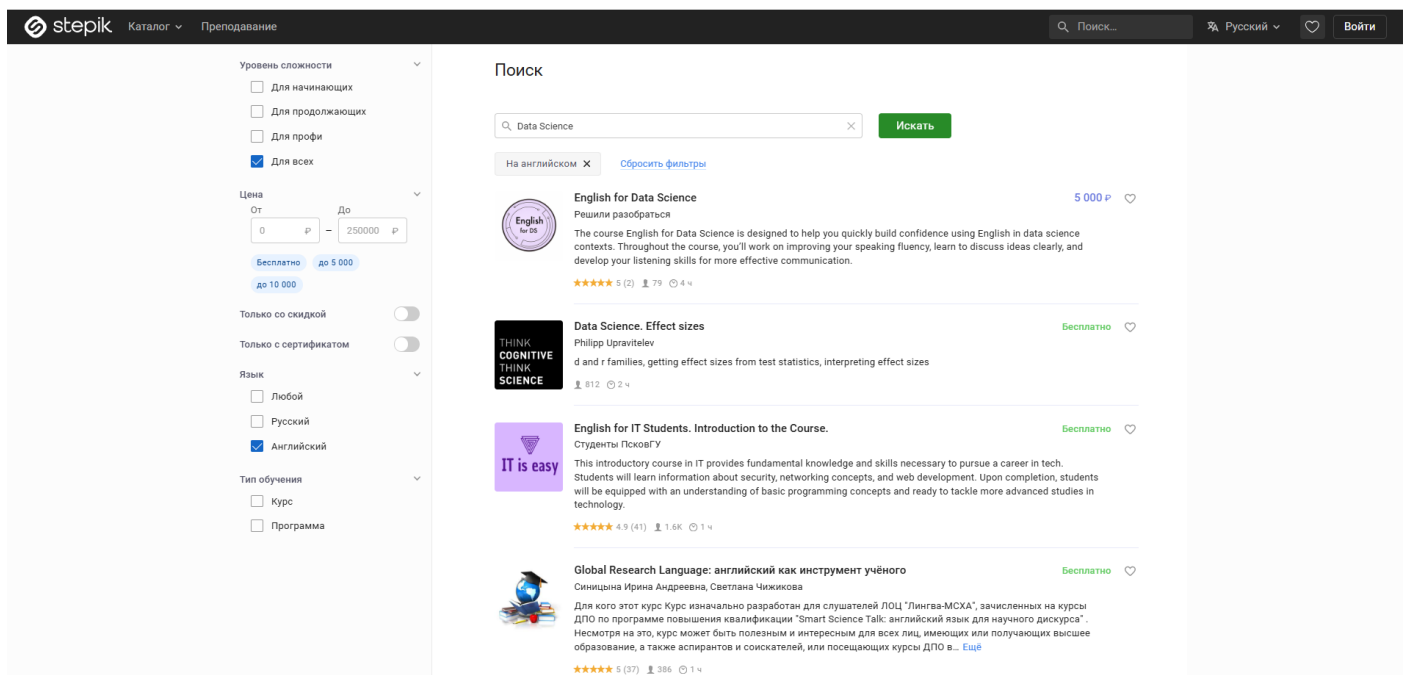


Рисунок 16 – Введение нового запроса «Data Science»

Тест №7: Несуществующий запрос с фильтрами

Данный тест проверяет корректность обработки негативного сценария, верифицируя появление сообщения «ничего не найдено» при вводе бессмысленного набора символов с примененными фильтрами.

Результаты действий представлены на рисунках (см. рисунки 17, 18).

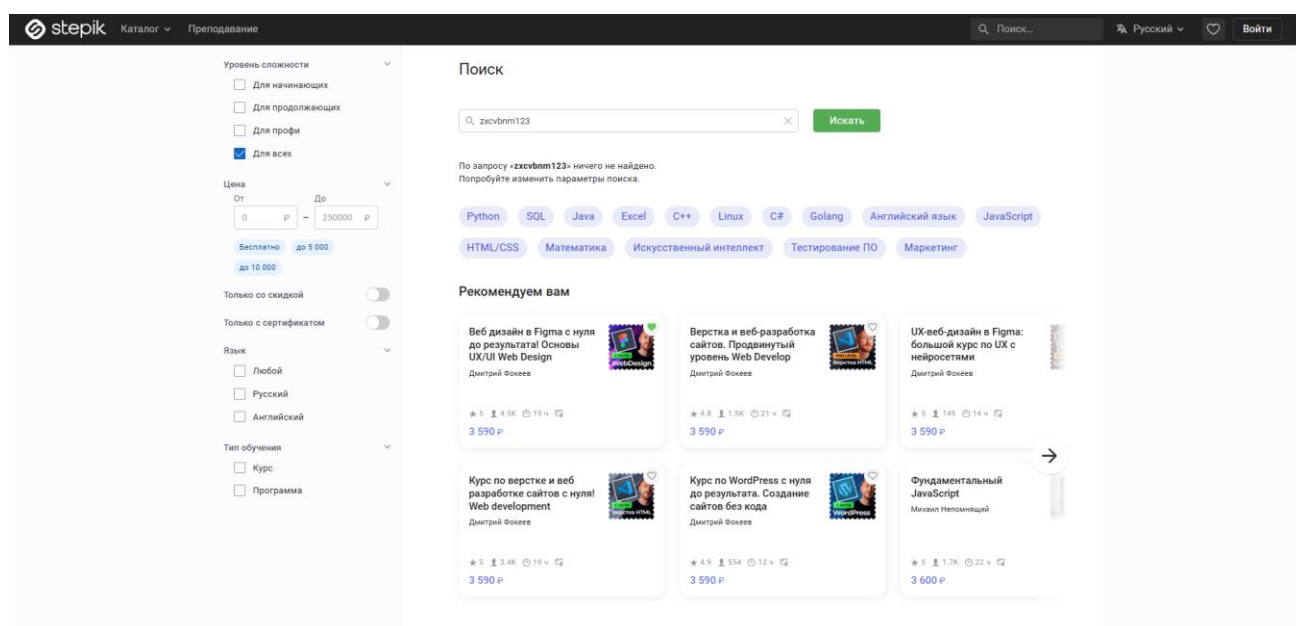


Рисунок 17 – Введение в поиск запрос «zxscvbnm123»

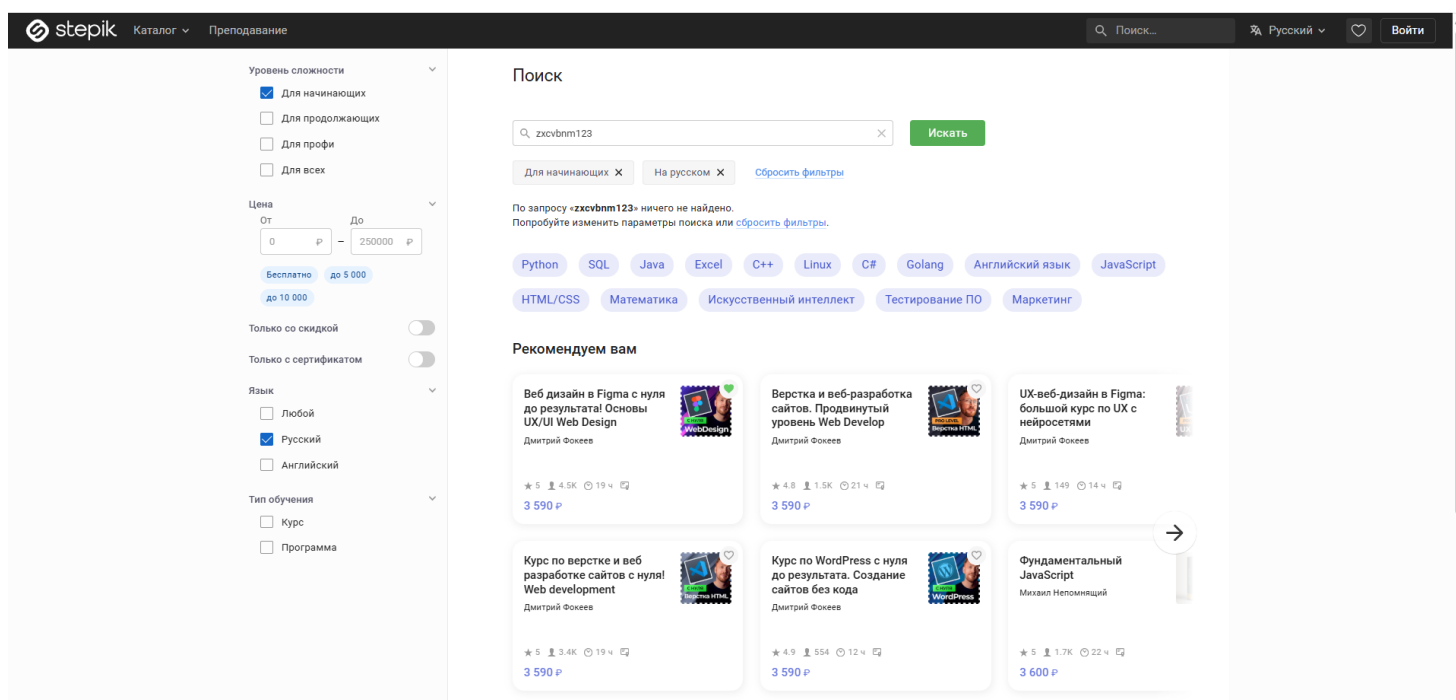


Рисунок 18 – Применение чекбоксов «Русский» и «Для начинающих»

Тест №8: Поиск по автору с переходом в профиль

Данный тест проверяет точность поиска курсов конкретного преподавателя и работоспособность ссылок, ведущих из карточки курса в личный профиль автора для просмотра отзывов.

Результаты действий представлены на рисунках (см. рисунки 19 – 22).

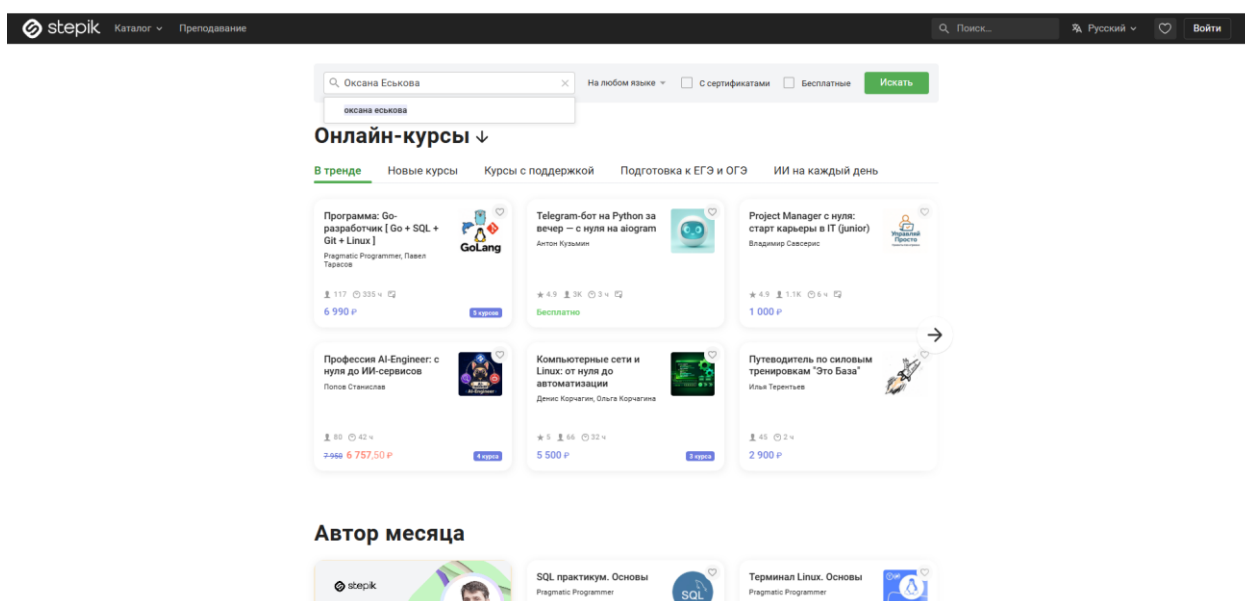


Рисунок 19 – Введение в поиск запрос «Оксана Еськова»

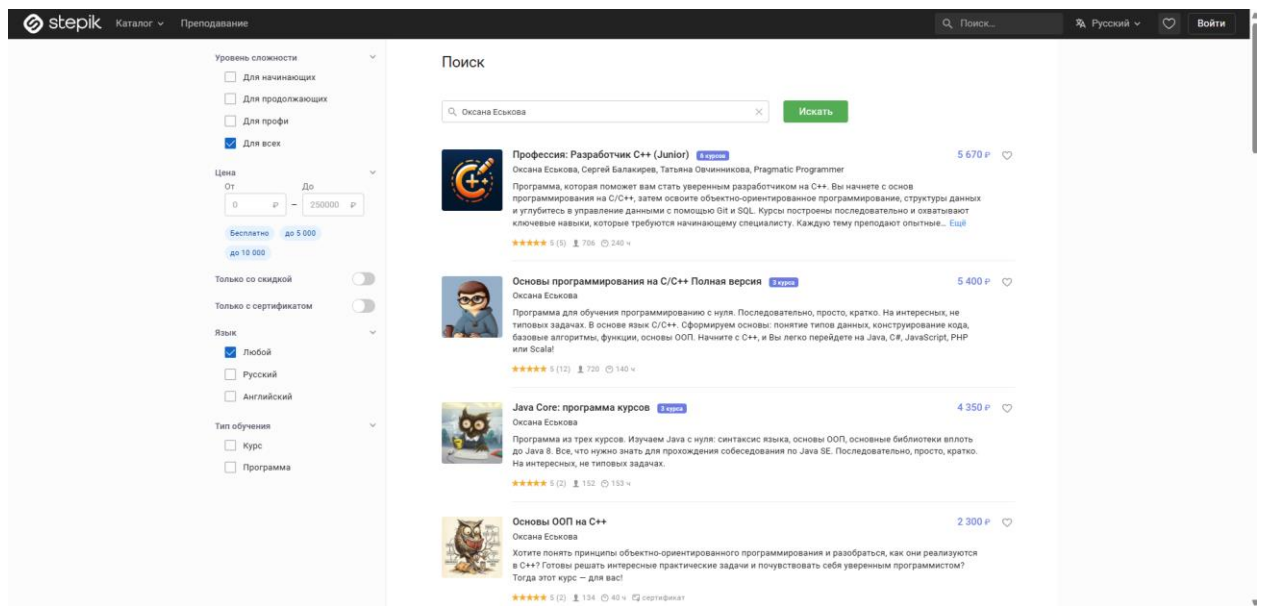


Рисунок 20 – Показ всех доступных курсов

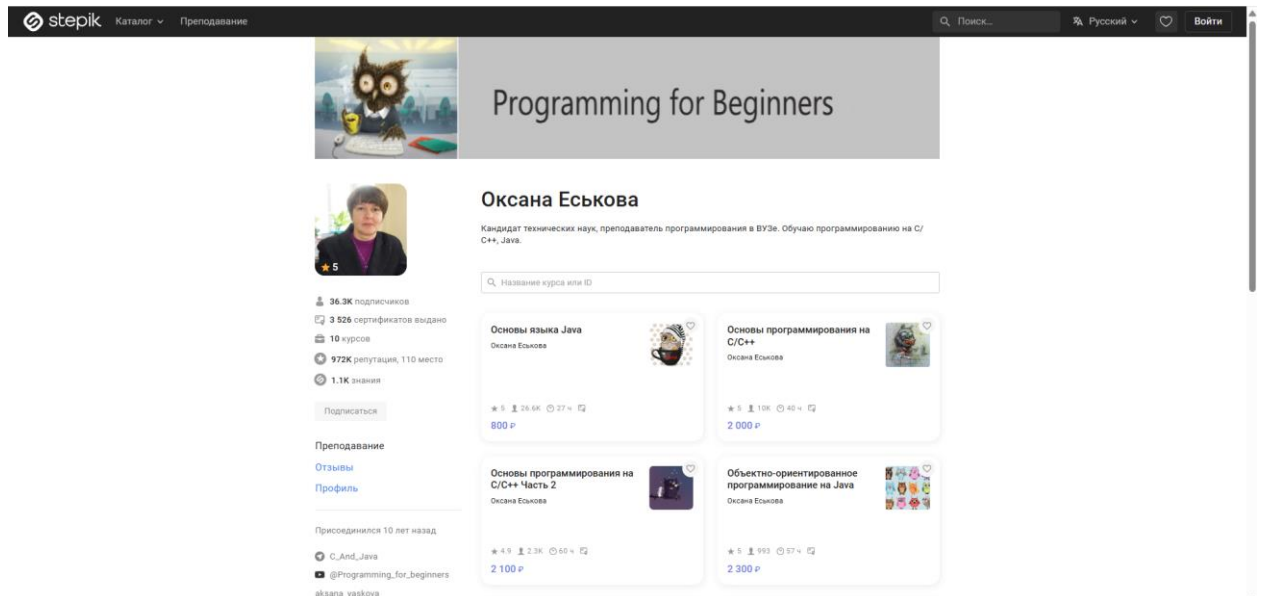


Рисунок 21 – Переход в профиль пользователя «Оксана Еськова»

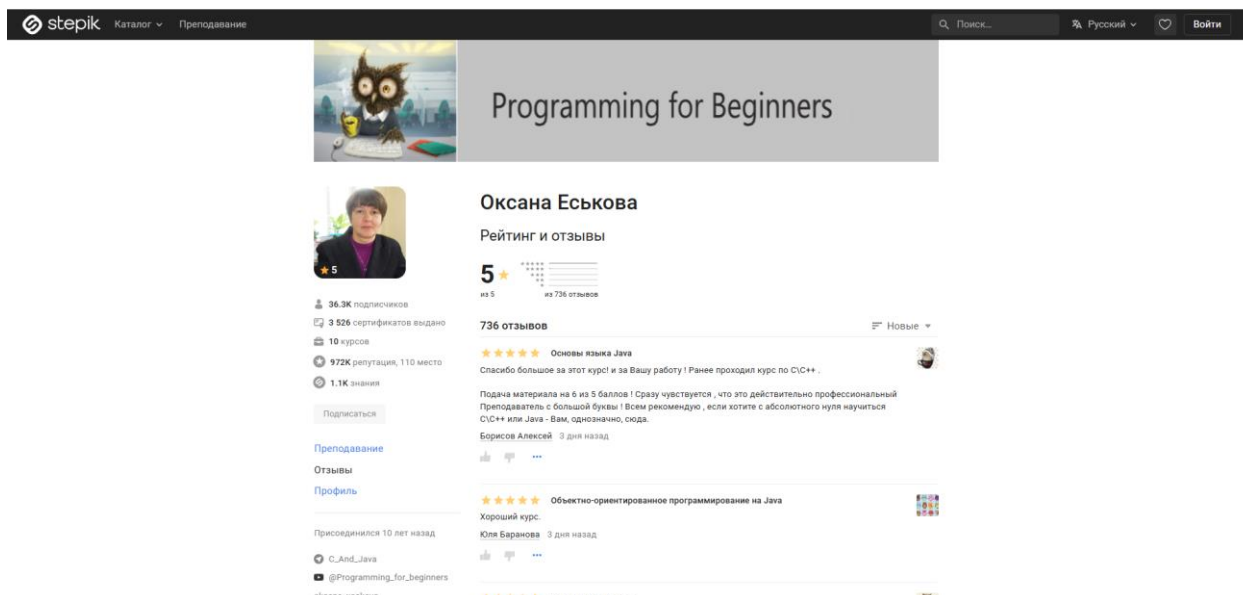


Рисунок 22 – Переход во вкладку «Отзывы»

Тест №9: Фильтрация акционных курсов

Данный тест верифицирует работу тумблера «со скидкой», проверяя, что в результатах выдачи отображаются карточки курсов, содержащие визуальный элемент зачеркнутой старой цены.

Результаты действий представлены на рисунках (см. рисунки 23, 24).

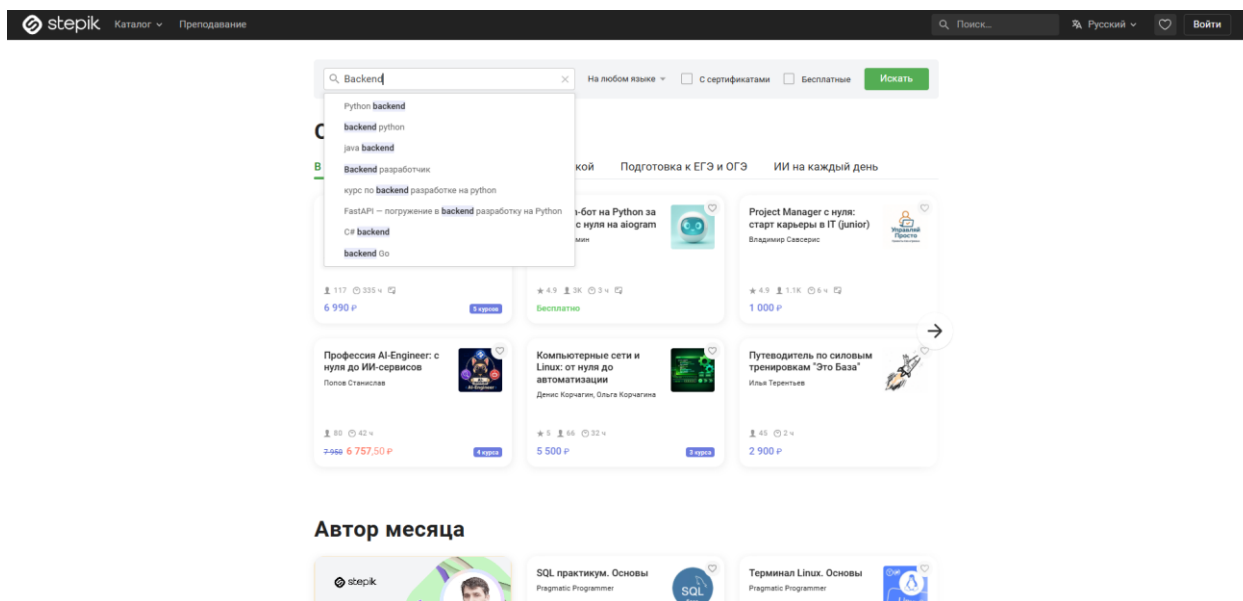


Рисунок 23 – Введение в поиск запрос «Backend»

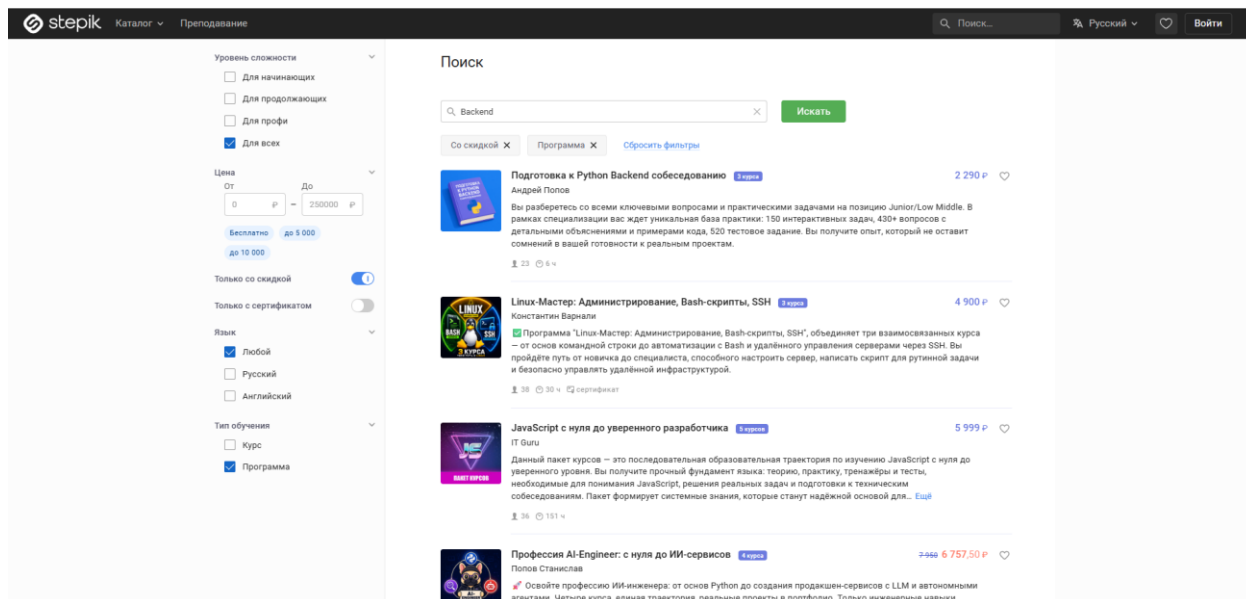


Рисунок 24 – Применение чекбоксов «Только со скидкой» и «Программа»

Тест №10: Добавление курса в избранное из поиска

Данный тест показывает интерактивное взаимодействие с компонентами карточки, проверяя возможность добавления курса в список желаемого (wishlist) и мгновенное изменение состояния иконки «сердечко».

Результаты действий представлены на рисунках (см. рисунки 25 – 27).

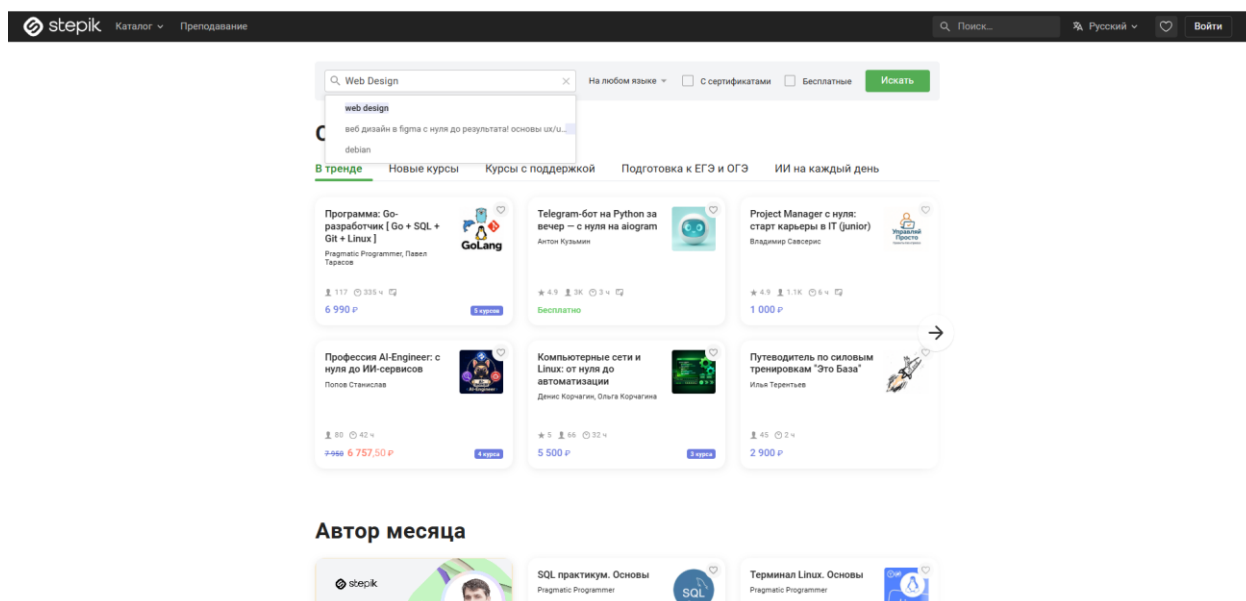


Рисунок 25 – Введение в поиск запрос «Web Design»

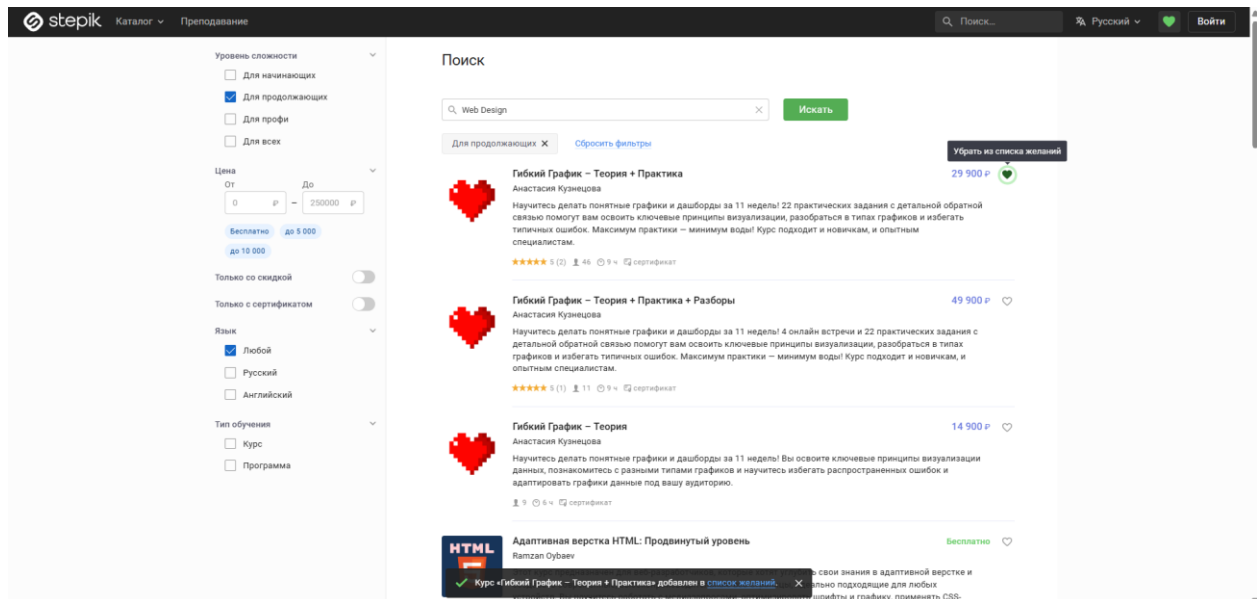


Рисунок 26 – Добавление первого курса в избранное

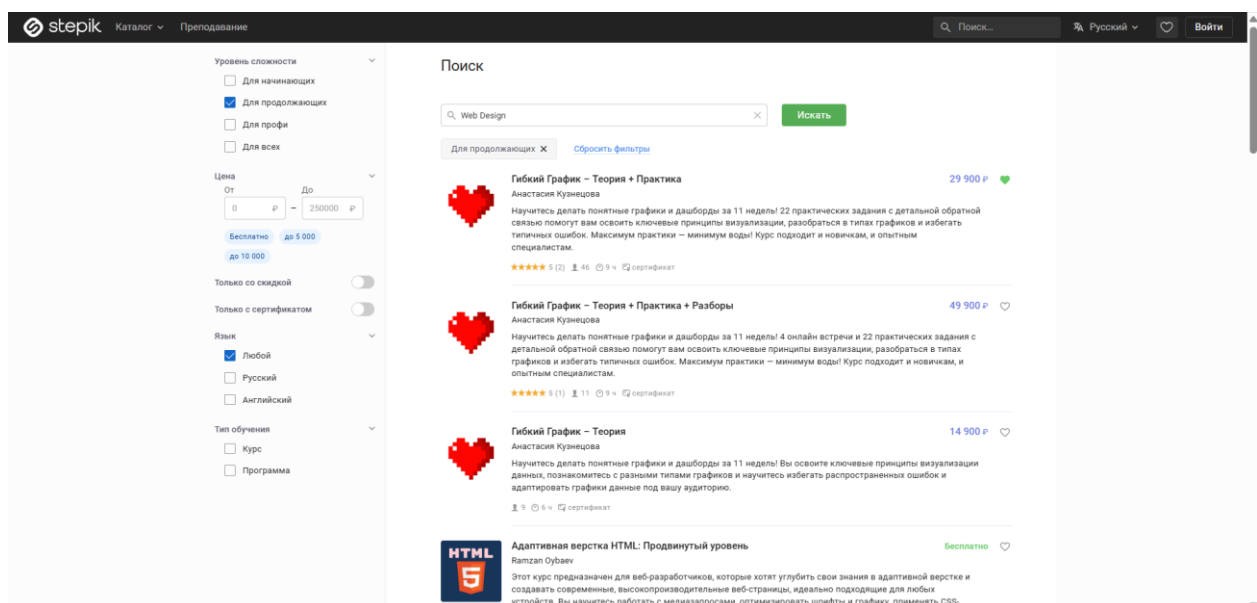


Рисунок 27 – Показ того, что первый курс в избранном (сердечко стало закрашенным)

Тесты поделены на два блока: поиск и фильтрация.

Результаты выполнения первой группы тестов представлены на рис. 28:

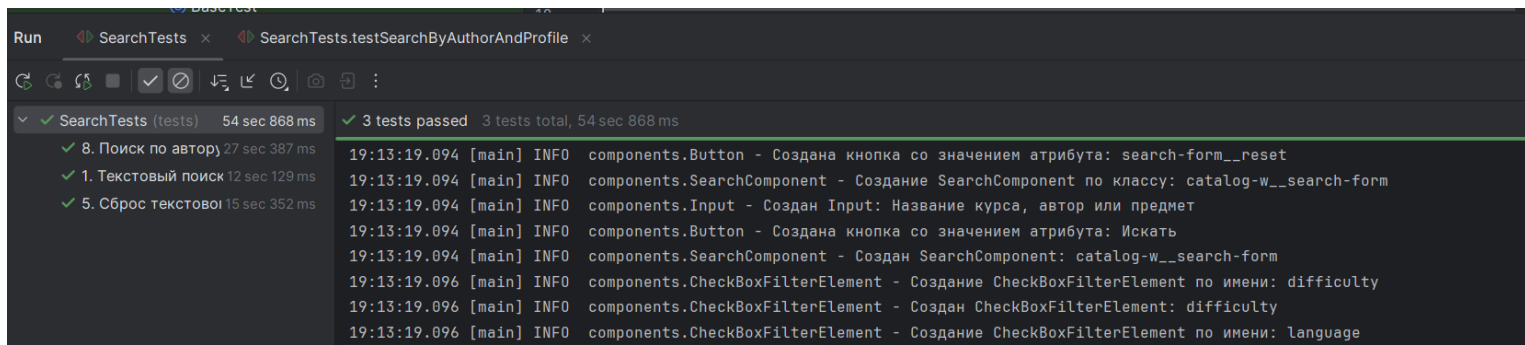


Рисунок 28 – Результаты первого блока

Результаты выполнения второй группы тестов представлены на рис. 29

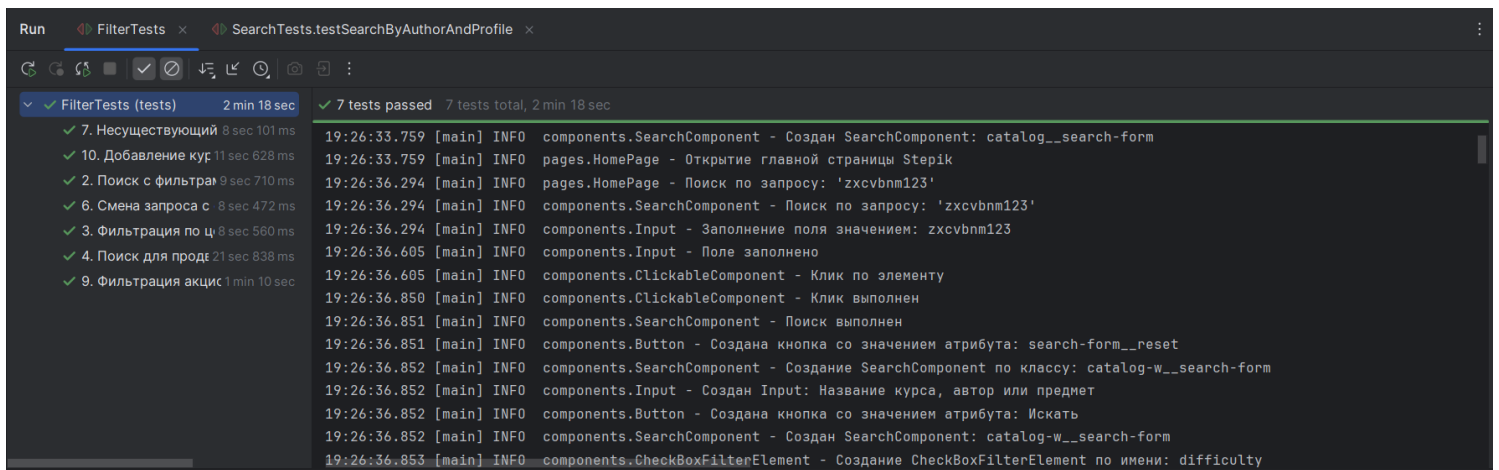


Рисунок 29 – Результаты второго блока

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет **components** (UI элементы):

1. **BaseComponent**: Абстрактный базовый класс, представляющий собой обертку над **SelenideElement**. Содержит общую логику поиска элементов по XPath [5] и методы доступа к ним.
2. **ClickableComponent**: Интерфейс, определяющий стандартное поведение для кликабельных элементов с обработкой ожидания видимости и логированием ошибок.
3. **Button**: Специализированный класс для работы с кнопками. Реализует интерфейс **ClickableComponent**. Содержит методы создания кнопки по названию (**byName**) и по CSS-классу (**byClass**).
4. **Input**: Специализированный класс для работы с полями ввода. Содержит методы создания поля по **placeholder** (**byPlaceHolder**), заполнения поля (**fill**) и получения значения (**getValue**).
5. **CheckBox**: Специализированный класс для работы с чекбоксами. Реализует интерфейс **ClickableComponent**. Содержит метод создания чекбокса по значению **data-qa-value** (**byValue**).
6. **CheckBoxFilterElement**: Класс для работы с группой чекбоксов внутри блока фильтрации. Содержит методы создания фильтра по имени (**byName**) и получения чекбокса по значению внутри группы (**getCheckBoxByValue**).

7. SearchComponent: Составной компонент, объединяющий строку поиска и кнопку отправки запроса. Содержит методы создания по CSS-классу (byClass), выполнения поиска (search) и получения текущего значения поля (getCurrentInputValue).
8. FilterComponent: Составной компонент, инкапсулирующий логику работы со всеми фильтрами на странице поиска: фильтрация по цене, сложности, языку и типу курса. Содержит методы применения чекбоксов (filterByCheckBox), ценового фильтра (filterByMinPrice, filterByMaxPrice), тумблеров (filterByToggler) и проверки состояния чекбокса (isCheckBoxSelected).
9. PriceFilterElement: Компонент для работы с ценовым фильтром. Содержит поля ввода минимальной и максимальной цены. Содержит методы получения полей ввода (getminValueInput, getMaxValueInput).
10. ToggleFilterElement: Класс для работы с тумблерами-переключателями. Содержит методы создания по имени (byName), переключения (toggle) и проверки активности (isActive).
11. CourseCardItem: Класс для работы с отдельной карточкой курса в поисковой выдаче. Реализует методы добавления в избранное (addToWishlist), клика по первому автору (clickFirstAuthor), получения заголовка (getTitle), проверки старой цены (hasOldPrice) и проверки наличия в избранном (isFavorite).
12. CourseCardText: Класс для работы с текстовым элементом внутри карточки курса (название или автор). Реализует интерфейс ClickableComponent. Содержит метод получения текста (getText).

Пакет pages (Page Objects):

1. BasePage: Абстрактная основа для всех страниц, содержащая общие параметры ожидания и методы инициализации.
2. HomePage: Описывает главную страницу каталога и инициализирует процесс поиска.

3. `SearchResultsPage`: Центральный класс, управляющий результатами поиска, применением фильтров и навигацией по страницам (пагинацией).

4. `CoursePage` и `ProfilePage`: Классы для взаимодействия со страницей конкретного курса и профилем автора соответственно.

Пакет `helpers` и `tests`:

1. `WindowHandler`: Вспомогательный класс для управления окнами и вкладками браузера.

2. `BaseTest`: Класс конфигурации, отвечающий за настройку драйвера перед запуском тестов и закрытие сессии после их завершения.

3. `SearchTests`: Основной тестовый класс, содержащий сценарии текстового поиска, проверки пустых запросов, поиска со спецсимволами и навигации по результатам выдачи.

4. `FilterTests`: Тестовый класс, объединяющий сценарии проверки различных фильтров (цена, язык, сложность, акции). Вынесение этих тестов в отдельный класс позволило логически разделить функционал поиска и фильтрации.

5. `TestConstants`: Класс-хранилище статических констант. В него вынесены все «магические числа» (таймауты, ожидаемые тексты, тестовые наборы данных), что обеспечивает удобство редактирования тестов без изменения программной логики.

2.2. UML-диаграмма классов

На рисунке (см. рис. 30) приведена UML-диаграмма, отображающая структуру классов, их иерархию наследования, реализацию интерфейсов и взаимосвязи между компонентами и страницами.

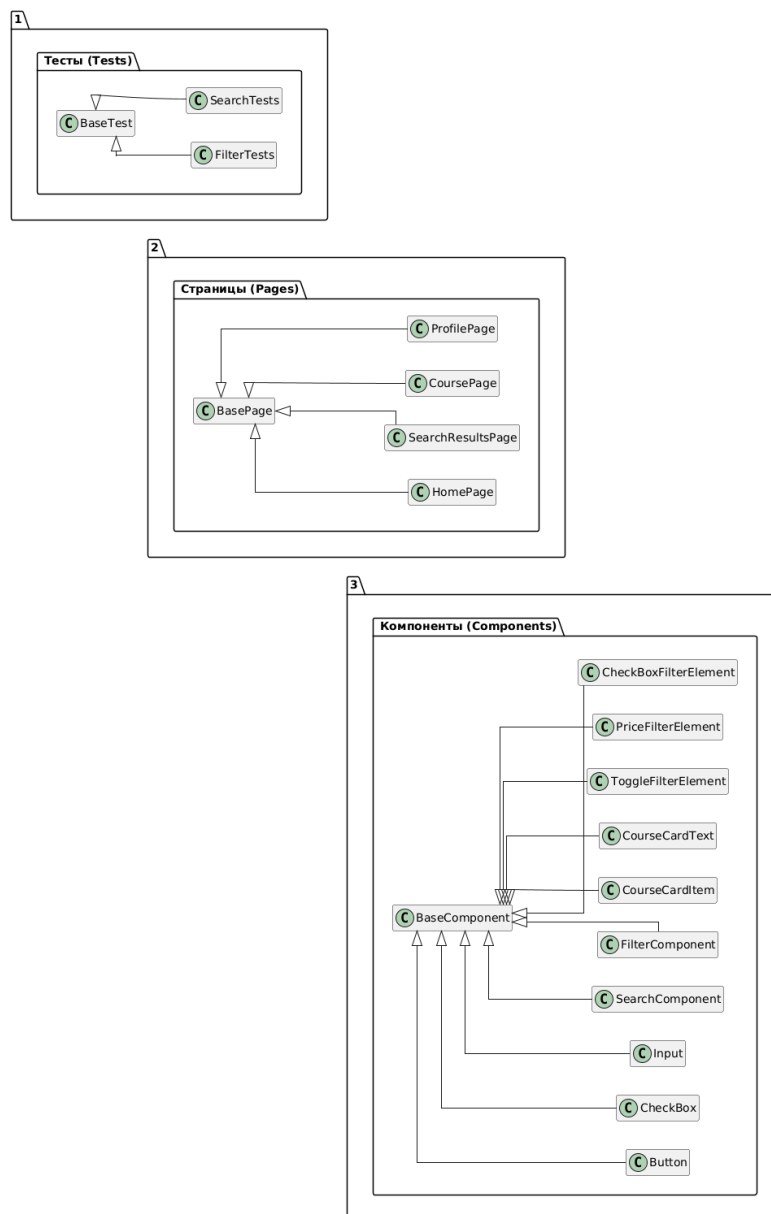


Рисунок 30 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Взаимодействие между классами построено по иерархическому принципу. Тестовые методы обращаются к методам классов страниц (Page Objects). Страницы, в свою очередь, делегируют управление специфическими блоками интерфейса составным компонентам (Components).

Например, при вызове метода `applyMinPriceFilter()` в классе `SearchResultsPage` происходит обращение к вложенному объекту `FilterComponent`, который взаимодействует с объектом `Input`. Такой

подход позволяет изменять локаторы элементов в одном месте (в классе компонента) без необходимости править код тестов.

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В рамках работы над проектом автоматизации тестирования образовательной платформы Stepik мной была выполнена часть работы, связанная с проектированием и реализацией архитектуры тестового фреймворка на основе паттерна Page Object Model (POM). Как тимлид, я также координировала работу команды, распределяла задачи и следила за соблюдением архитектурных принципов.

Конкретные задачи, выполненные лично:

1. Реализация Page Object Model

Мной были разработаны все классы страниц (pages), обеспечивающие взаимодействие с основными элементами интерфейса платформы Stepik:

- HomePage - главная страница каталога. Реализованы методы открытия страницы и выполнения поискового запроса.
- SearchResultsPage - страница результатов поиска. Реализованы методы работы с фильтрацией, пагинацией, карточками курсов, очисткой поиска и повторным поиском.
- CoursePage - страница отдельного курса. Реализован метод получения заголовка курса для проверки корректности перехода.
- ProfilePage - страница профиля пользователя. Реализован метод открытия вкладки «Отзывы» и проверки загрузки соответствующего заголовка.

2. Организация логирования

Мной была настроена система логирования на основе Log4j2:

- Добавлены зависимости в pom.xml.
- Создан конфигурационный файл log4j2.xml с настройкой вывода логов в консоль и в файл.
- Реализовано логирование во всех классах pages (инициализация страниц, выполнение действий, результаты проверок).

- Логирование позволило отслеживать ход выполнения тестов и оперативно выявлять причины падений.

3. Создание чек-листа

Мной был разработан подробный чек-лист из 10 тестовых сценариев, охватывающих ключевые пользовательские пути при работе с поиском и фильтрацией курсов на платформе Stepik. Чек-лист включает:

- название теста;
- входные данные;
- пошаговое описание действий пользователя;
- ожидаемый результат.

4. Подготовка и финализация README

Мной был подготовлен и финализирован файл README.md для репозитория проекта, в котором описаны:

- назначение и цели проекта;
- технологический стек;
- структура проекта;
- список и описание 10 тестов;
- инструкция по установке и запуску тестов;
- распределение ролей в команде.

5. Приведение классов к единому порядку

Мной была проведена работа по приведению всех классов к единому стандарту расположения элементов в папке pages:

- static поля (логгер, константы XPath)
- instance поля (элементы страницы, компоненты)
- Конструкторы
- static методы
- public методы
- protected методы
- private методы

Это обеспечило единообразие кодовой базы и упростило её поддержку.

6. Вынесение Selenide элементов в переменные

В ходе рефакторинга все прямые вызовы `$x()` и `$()` были вынесены в переменные класса в папке `pages` с модификатором `private final`. Это позволило:

- избежать дублирования локаторов;
- упростить изменение XPath при необходимости;
- повысить читаемость и поддерживаемость кода.

7. Удаление неиспользуемых методов и импортов

Мной был проведён анализ кодовой базы и удалены все неиспользуемые методы и неиспользуемые импорты в папке `pages`, что позволило сократить объём кода и повысить его чистоту.

8. Добавление документации к классам `pages` и `BaseTest`

Для всех классов в пакете `pages` и для класса `BaseTest` была добавлена JavaDoc-документация, описывающая:

- назначение класса;
- основные методы;
- параметры и возвращаемые значения.

Это облегчило понимание кода другими участниками команды и создало основу для дальнейшего развития проекта.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе реализации учебной практики был сформирован технологический стек, обеспечивающий высокую надежность и удобство поддержки разрабатываемого решения. В качестве основного языка программирования был выбран Java, что позволило в полной мере реализовать объектно-ориентированный подход и выстроить четкую архитектуру проекта с разделением ответственности между классами страниц, компонентов и тестов. Управление зависимостями и сборка проекта осуществлялись с помощью инструмента Maven.

Для автоматизации взаимодействия с интерфейсом платформы Stepik применялся фреймворк Selenide. Выбор данного инструмента обусловлен его эффективностью при работе с динамическими элементами, а также наличием встроенных механизмов неявных ожиданий, что существенно повышает стабильность тестов. Структура тестовых сценариев и их запуск реализованы на базе библиотеки JUnit 5, а для мониторинга хода выполнения и оперативного выявления ошибок в проект интегрирована система логирования Log4j2.

Применение выбранных технологий позволило создать масштабируемый тестовый фреймворк и успешно автоматизировать проверку ключевых функций поиска и фильтрации каталога Stepik.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики были успешно достигнуты все поставленные цели и реализованы задачи по созданию системы автоматизированного тестирования платформы Stepik. В рамках первой задачи был проведен детальный анализ функциональных возможностей тестируемой платформы, что позволило выявить наиболее критические пользовательские пути, связанные с поиском и подбором образовательного контента. Итогом этого этапа стал сформированный чек-лист из десяти тестовых сценариев, которые охватывают широкий спектр функций: от базового текстового поиска и навигации по страницам до сложных комбинаций фильтров по языку, сложности и стоимости курсов. Проведенный анализ обеспечил полное покрытие требований к качеству поиска и корректности отображения результатов выдачи.

Вторая задача по разработке программной архитектуры проекта была решена путем внедрения объектно-ориентированного подхода на языке Java с использованием паттерна Page Object Model (POM). Реализованная архитектура позволила эффективно изолировать логику тестовых сценариев от деталей реализации пользовательского интерфейса. Благодаря применению компонентно-ориентированного подхода были созданы универсальные классы для работы с фильтрами и строкой поиска, что значительно повысило гибкость системы и упростило ее дальнейшую поддержку. Структура проекта, разделенная на уровни компонентов (UI elements) и страниц (Page Objects), обеспечивает высокую читаемость кода и возможность повторного использования отдельных модулей в будущих разработках.

Третья задача, включающая программную реализацию тестов и подготовку документации, была выполнена с применением современного стека технологий, включая библиотеки Selenide и JUnit5. Использование фреймворка Selenide позволило реализовать надежные и стабильные автотесты с автоматическим управлением ожиданиями элементов. Для обеспечения мониторинга выполнения проверок в проект была успешно

интегрирована система логирования Log4j2, предоставляющая детальную информацию о каждом шаге выполнения сценариев в режиме реального времени. В качестве завершающего этапа была подготовлена техническая документация, включающая детальные комментарии и визуализацию структуры системы с помощью UML-диаграмм классов, что наглядно демонстрирует иерархию и взаимосвязи программных модулей.

В целом, работа над проектом позволила закрепить навыки проектирования архитектуры ПО и освоить инструменты промышленной автоматизации тестирования. Цель практики по обеспечению надежности функционала поиска была полностью достигнута. Все разработанные программные классы приведены к единому стандарту кодирования, очищены от неиспользуемого кода и снабжены необходимой документацией. В завершение работы над проектом все исходные коды, файлы были выложены в GitHub-репозитории [6], что обеспечивает сохранность наработок, версиюность кода и возможность совместного доступа к результатам тестирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fowler, M. PageObject [Электронный ресурс].
URL: <https://martinfowler.com/bliki/PageObject.html> (дата обращения: 07.07.2026).
2. Selenide: Concise UI Tests in Java [Электронный ресурс].
URL: <https://selenide.org/> (дата обращения: 08.07.2026).
3. JUnit 5 User Guide [Электронный ресурс].
URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. Apache Log4j2 Manual [Электронный ресурс]. URL: <https://logging.apache.org/log4j/2.x/manual/index.html> (дата обращения: 10.07.2024).
5. Язык запросов XPath — руководство MSiter [Электронный ресурс].
URL: <https://msiter.ru/tutorials/xpath> (дата обращения: 07.07.2026).
6. <https://github.com/vyixwq/Stepik-practice>